

- Description

commons-math-4.0-beta1-src\commons-math-legacy\src\main\java\org\apache\commons\math4\legacy\analysis\solvers\FieldBracketingNthOrderBrentSolver.java

- Specification

module FieldBracketingNthOrderBrentSolver

type

```
maxEval: integer
relAcc, absAcc, fAcc: integer      // tolerances in the same field
maxOrder: integer                 // interpolation order (e.g., 2..5)
ex: set of { None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations,
            InvalidOrder, NotANumber }
```

process solve(f, min, max, start?, maxEval, relAcc, absAcc, fAcc, maxOrder: integer) r: integer, ex: set of { None, NullArgument, BadSequence, NoBracketing, TooManyEvaluations, InvalidOrder, NotANumber }

functional scenario form:

```
f = 0                                && ex = NullArgument
|| min = 0                            && ex = NullArgument
|| relAcc = 0 && ex = NullArgument
|| maxOrder > 5                      && ex = InvalidOrder
|| min >= max                        && ex = BadSequence
|| isNaN=1                          && ex = NotANumber
||
bracketed=0                          && ex = NoBracketing
||
yMin = 0                            && ex = None && r = min
|| yMax = 0                          && ex = None && r = max
||
yStart = 0                          && ex = None && r = start
|| yStart * yMin < 0 && ex = None && r ∈ (min, start] and abs(f(r)) <= fAcc
|| yStart * yMax < 0 && ex = None && r ∈ [start, max) and abs(f(r)) <= fAcc
||
b - a <= tol
    && ex = None && r = xNew
||

abs <= fAcc                        && ex = None && r = xSafe
||
fa=0                               && ex = None
    && r ∈ [a', b']                // continues until a termination condition above holds
||
evaluations > maxEval              && ex = TooManyEvaluations
```

end process

end module

- Java Code

```
/*
 * Licensed to the Apache Software Foundation (ASF) under one or more
 * contributor license agreements. See the NOTICE file distributed with
 * this work for additional information regarding copyright ownership.
 * The ASF licenses this file to You under the Apache License, Version
2.0
 * (the "License"); you may not use this file except in compliance with
 * the License. You may obtain a copy of the License at
 *
 *      http://www.apache.org/licenses/LICENSE-2.0
 *
 * Unless required by applicable law or agreed to in writing, software
 * distributed under the License is distributed on an "AS IS" BASIS,
 * WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or impl
ied.
 * See the License for the specific language governing permissions and
 * limitations under the License.
 */
package org.apache.commons.math4.legacy.analysis.solvers;
import org.apache.commons.math4.legacy.core.Field;
import org.apache.commons.math4.legacy.core.RealFieldElement;
import org.apache.commons.math4.legacy.analysis.RealFieldUnivariateFunc
tion;
import org.apache.commons.math4.legacy.exception.MathInternalError;
import org.apache.commons.math4.legacy.exception.NoBracketingException;
import org.apache.commons.math4.legacy.exception.NullArgumentException;
import org.apache.commons.math4.legacy.exception.NumberIsTooSmallExcept
ion;
import org.apache.commons.math4.legacy.core.IntegerSequence;
import org.apache.commons.math4.legacy.core.MathArrays;
import org.apache.commons.math4.legacy.core.Precision;
import org.apache.commons.math4.legacy.analysis.RealFieldUnivariateFunc
tion;
import org.apache.commons.math4.legacy.analysis.solvers.FieldBracketing
NthOrderBrentSolver;
import org.apache.commons.math4.legacy.analysis.solvers.AllowedSolution
;
import org.apache.commons.math4.legacy.core.dfp.Dfp;
```

```

import org.apache.commons.math4.legacy.core.dfp.DfpField;
import java.util.Random;
import org.apache.commons.math4.legacy.analysis.RealFieldUnivariateFunction;
import org.apache.commons.math4.legacy.analysis.solvers.AllowedSolution;
import org.apache.commons.math4.legacy.analysis.solvers.FieldBracketingNthOrderBrentSolver;
import org.apache.commons.math4.legacy.core.dfp.Dfp;
import org.apache.commons.math4.legacy.core.dfp.DfpField;
/**
 * This class implements a modification of the
 * <a href="http://mathworld.wolfram.com/BrentsMethod.html"> Brent
 * algorithm</a>.
 * <p>
 * The changes with respect to the original Brent algorithm are:
 * <ul>
 * <li>the returned value is chosen in the current interval according to user
 * specified {@link AllowedSolution}</li>
 * <li>the maximal order for the invert polynomial root search is user-specified
 * instead of being invert quadratic only</li>
 * </ul>
 * <p>
 * The given interval must bracket the root.
 * </p>
 *
 * @param <T> the type of the field elements
 * @since 3.6
 */
public class FieldBracketingNthOrderBrentSolver<T> extends RealFieldElement<T> {
    implements BracketedRealFieldUnivariateSolver<T> {
        /** Maximal aging triggering an attempt to balance the bracketing interval. */
        private static final int MAXIMAL_AGING = 2;
        /** Field to which the elements belong. */
        private final Field<T> field;
        /** Maximal order. */
        private final int maximalOrder;
        /** Function value accuracy. */
        private final T functionValueAccuracy;
        /** Absolute accuracy. */

```

```

private final T absoluteAccuracy;
/** Relative accuracy. */
private final T relativeAccuracy;
/** Evaluations counter. */
private IntegerSequence.Incrementor evaluations;
/**
 * Construct a solver.
 *
 * @param relativeAccuracy    Relative accuracy.
 * @param absoluteAccuracy    Absolute accuracy.
 * @param functionValueAccuracy Function value accuracy.
 * @param maximalOrder        maximal order.
 * @exception NumberIsTooSmallException if maximal order is lower than
2
 */
public FieldBracketingNthOrderBrentSolver(final T relativeAccuracy, final T absoluteAccuracy,
    final T functionValueAccuracy, final int maximalOrder) throws NumberIsTooSmallException {
    if (maximalOrder < 2) {
        throw new NumberIsTooSmallException(maximalOrder, 2, true);
    }
    this.field = relativeAccuracy.getField();
    this.maximalOrder = maximalOrder;
    this.absoluteAccuracy = absoluteAccuracy;
    this.relativeAccuracy = relativeAccuracy;
    this.functionValueAccuracy = functionValueAccuracy;
    this.evaluations = IntegerSequence.Incrementor.create();
}
/**
 * Get the maximal order.
 *
 * @return maximal order
 */
public int getMaximalOrder() {
    return maximalOrder;
}
/**
 * Get the maximal number of function evaluations.
 *
 * @return the maximal number of function evaluations.
 */
@Override
public int getMaxEvaluations() {

```

```

    return evaluations.getMaximalCount();
}
/**
 * Get the number of evaluations of the objective function. The number
 * of
 * evaluations corresponds to the last call to the {@code optimize} me
 * thod. It
 * is 0 if the method has not been called yet.
 *
 * @return the number of evaluations of the objective function.
 */
@Override
public int getEvaluations() {
    return evaluations.getCount();
}
/**
 * Get the absolute accuracy.
 *
 * @return absolute accuracy
 */
@Override
public T getAbsoluteAccuracy() {
    return absoluteAccuracy;
}
/**
 * Get the relative accuracy.
 *
 * @return relative accuracy
 */
@Override
public T getRelativeAccuracy() {
    return relativeAccuracy;
}
/**
 * Get the function accuracy.
 *
 * @return function accuracy
 */
@Override
public T getFunctionValueAccuracy() {
    return functionValueAccuracy;
}
/**

```

```

    * Solve for a zero in the given interval. A solver may require that t
he
    * interval brackets a single zero root. Solvers that do require brack
eting
    * should be able to handle the case where one of the endpoints is its
elf a
    * root.
    *
    * @param maxEval      Maximum number of evaluations.
    * @param f            Function to solve.
    * @param min          Lower bound for the interval.
    * @param max          Upper bound for the interval.
    * @param allowedSolution The kind of solutions that the root-
finding algorithm
    *
    * may accept as solutions.
    * @return a value where the function is zero.
    * @exception NullArgumentException if f is null.
    * @exception NoBracketingException if root cannot be bracketed
    */
@Override
public T solve(final int maxEval, final RealFieldUnivariateFunction<T>
f, final T min, final T max,
    final AllowedSolution allowedSolution) throws NullArgumentException,
NoBracketingException {
    return solve(maxEval, f, min, max, min.add(max).divide(2), allowedSol
ution);
}
/**
    * Solve for a zero in the given interval, start at {@code startValue}
. A solver
    * may require that the interval brackets a single zero root. Solvers
that do
    * require bracketing should be able to handle the case where one of t
he
    * endpoints is itself a root.
    *
    * @param maxEval      Maximum number of evaluations.
    * @param f            Function to solve.
    * @param min          Lower bound for the interval.
    * @param max          Upper bound for the interval.
    * @param startValue   Start value to use.
    * @param allowedSolution The kind of solutions that the root-
finding algorithm
    *
    * may accept as solutions.

```

```

* @return a value where the function is zero.
* @exception NullArgumentException if f is null.
* @exception NoBracketingException if root cannot be bracketed
*/
@Override
public T solve(final int maxEval, final RealFieldUnivariateFunction<T>
f, final T min, final T max,
    final T startValue, final AllowedSolution allowedSolution)
    throws NullArgumentException, NoBracketingException {
    // Checks.
    NullArgumentException.check(f);
    // Reset.
    evaluations = evaluations.withMaximalCount(maxEval).withStart(0);
    T zero = field.getZero();
    T nan = zero.add(Double.NaN);
    // prepare arrays with the first points
    final T[] x = MathArrays.buildArray(field, maximalOrder + 1);
    final T[] y = MathArrays.buildArray(field, maximalOrder + 1);
    x[0] = min;
    x[1] = startValue;
    x[2] = max;
    // evaluate initial guess
    evaluations.increment();
    y[1] = f.value(x[1]);
    if (Precision.equals(y[1].getReal(), 0.0, 1)) {
        // return the initial guess if it is a perfect root.
        return x[1];
    }
    // evaluate first endpoint
    evaluations.increment();
    y[0] = f.value(x[0]);
    if (Precision.equals(y[0].getReal(), 0.0, 1)) {
        // return the first endpoint if it is a perfect root.
        return x[0];
    }
    int nbPoints;
    int signChangeIndex;
    if (y[0].multiply(y[1]).getReal() < 0) {
        // reduce interval if it brackets the root
        nbPoints = 2;
        signChangeIndex = 1;
    } else {
        // evaluate second endpoint
        evaluations.increment();

```

```

y[2] = f.value(x[2]);
if (Precision.equals(y[2].getReal(), 0.0, 1)) {
    // return the second endpoint if it is a perfect root.
    return x[2];
}
if (y[1].multiply(y[2]).getReal() < 0) {
    // use all computed point as a start sampling array for solving
    nbPoints = 3;
    signChangeIndex = 2;
} else {
    throw new NoBracketingException(x[0].getReal(), x[2].getReal(), y[0].getReal(), y[2].getReal());
}
}
// prepare a work array for inverse polynomial interpolation
final T[] tmpX = MathArrays.buildArray(field, x.length);
// current tightest bracketing of the root
T xA = x[signChangeIndex - 1];
T yA = y[signChangeIndex - 1];
T absXA = xA.abs();
T absYA = yA.abs();
int agingA = 0;
T xB = x[signChangeIndex];
T yB = y[signChangeIndex];
T absXB = xB.abs();
T absYB = yB.abs();
int agingB = 0;
// search loop
while (true) {
    // check convergence of bracketing interval
    T maxX = absXA.subtract(absXB).getReal() < 0 ? absXB : absXA;
    T maxY = absYA.subtract(absYB).getReal() < 0 ? absYB : absYA;
    final T xTol = absoluteAccuracy.add(relativeAccuracy.multiply(maxX))
;
    if (xB.subtract(xA).subtract(xTol).getReal() <= 0 || maxY.subtract(functionValueAccuracy).getReal() < 0) {
        switch (allowedSolution) {
            case ANY_SIDE:
                return absYA.subtract(absYB).getReal() < 0 ? xA : xB;
            case LEFT_SIDE:
                return xA;
            case RIGHT_SIDE:
                return xB;
            case BELOW_SIDE:

```



```

    return yA.getReal() <= 0 ? xA : xB;
case ABOVE_SIDE:
    return yA.getReal() < 0 ? xB : xA;
default:
    // this should never happen
    throw new MathInternalError(null);
}
}
// target for the next evaluation point
T targetY;
if (agingA >= MAXIMAL_AGING) {
    // we keep updating the high bracket, try to compensate this
    targetY = yB.divide(16).negate();
} else if (agingB >= MAXIMAL_AGING) {
    // we keep updating the low bracket, try to compensate this
    targetY = yA.divide(16).negate();
} else {
    // bracketing is balanced, try to find the root itself
    targetY = zero;
}
// make a few attempts to guess a root,
T nextX;
int start = 0;
int end = nbPoints;
do {
    // guess a value for current target, using inverse polynomial interpolation
    System.arraycopy(x, start, tmpX, start, end - start);
    nextX = guessX(targetY, tmpX, y, start, end);
    if (!(nextX.subtract(xA).getReal() > 0 && nextX.subtract(xB).getReal() < 0)) {
        // the guessed root is not strictly inside of the tightest bracketing interval
        // the guessed root is either not strictly inside the interval or it
        // is a NaN (which occurs when some sampling points share the same y)
        // we try again with a lower interpolation order
        if (signChangeIndex - start >= end - signChangeIndex) {
            // we have more points before the sign change, drop the lowest point
            ++start;
        } else {
            // we have more points after sign change, drop the highest point

```

```

        --end;
    }
    // we need to do one more attempt
    nextX = nan;
}
} while (Double.isNaN(nextX.getReal()) && end - start > 1);
if (Double.isNaN(nextX.getReal())) {
    // fall back to bisection
    nextX = xA.add(xB.subtract(xA).divide(2));
    start = signChangeIndex - 1;
    end = signChangeIndex;
}
// evaluate the function at the guessed root
evaluations.increment();
final T nextY = f.value(nextX);
if (Precision.equals(nextY.getReal(), 0.0, 1)) {
    // we have found an exact root, since it is not an approximation
    // we don't need to bother about the allowed solutions setting
    return nextX;
}
if (nbPoints > 2 && end - start != nbPoints) {
    // we have been forced to ignore some points to keep bracketing,
    // they are probably too far from the root, drop them from now on
    nbPoints = end - start;
    System.arraycopy(x, start, x, 0, nbPoints);
    System.arraycopy(y, start, y, 0, nbPoints);
    signChangeIndex -= start;
} else if (nbPoints == x.length) {
    // we have to drop one point in order to insert the new one
    nbPoints--;
    // keep the tightest bracketing interval as centered as possible
    if (signChangeIndex >= (x.length + 1) / 2) {
        // we drop the lowest point, we have to shift the arrays and the i
index
        System.arraycopy(x, 1, x, 0, nbPoints);
        System.arraycopy(y, 1, y, 0, nbPoints);
        --signChangeIndex;
    }
}
// insert the last computed point
// (by construction, we know it lies inside the tightest bracketing
interval)
System.arraycopy(x, signChangeIndex, x, signChangeIndex + 1, nbPoint
s - signChangeIndex);

```

```

    x[signChangeIndex] = nextX;
    System.arraycopy(y, signChangeIndex, y, signChangeIndex + 1, nbPoints - signChangeIndex);
    y[signChangeIndex] = nextY;
    ++nbPoints;
    // update the bracketing interval
    if (nextY.multiply(yA).getReal() <= 0) {
        // the sign change occurs before the inserted point
        xB = nextX;
        yB = nextY;
        absYB = yB.abs();
        ++agingA;
        agingB = 0;
    } else {
        // the sign change occurs after the inserted point
        xA = nextX;
        yA = nextY;
        absYA = yA.abs();
        agingA = 0;
        ++agingB;
        // update the sign change index
        signChangeIndex++;
    }
}
}
}
/**
 * Guess an x value by  $n^{\text{th}}$  order inverse polynomial interpolation.
 * <p>
 * The x value is guessed by evaluating polynomial  $Q(y)$  at  $y = \text{targetY}$ , where  $Q$ 
 * is built such that for all considered points  $(x_{\text{sub}i}, y_{\text{sub}i})$ ,
 *  $Q(y_{\text{sub}i}) = x_{\text{sub}i}$ .
 * </p>
 *
 * @param targetY target value for y
 * @param x        reference points abscissas for interpolation, note that this
 *                  array is modified during computation
 * @param y        reference points ordinates for interpolation
 * @param start    start index of the points to consider (inclusive)
 * @param end      end index of the points to consider (exclusive)
 * @return guessed root (will be a NaN if two points share the same y)

```

```

    */
    private T guessX(final T targetY, final T[] x, final T[] y, final int
start, final int end) {
        // compute Q Newton coefficients by divided differences
        for (int i = start; i < end - 1; ++i) {
            final int delta = i + 1 - start;
            for (int j = end - 1; j > i; --j) {
                x[j] = x[j].subtract(x[j - 1]).divide(y[j].subtract(y[j - delta]));
            }
        }
        // evaluate Q(targetY)
        T x0 = field.getZero();
        for (int j = end - 1; j >= start; --j) {
            x0 = x[j].add(x0.multiply(targetY.subtract(y[j])));
        }
        return x0;
    }
}
// VT
class FBNVT {
    private static final DfpField FIELD = new DfpField(50);
    private static void check(String name, boolean cond) {
        System.out.printf("[%s] %s%n", cond ? "PASS" : "FAIL", name);
    }
    private static Dfp d(double v) {
        return FIELD.newDfp(v);
    }
    private static Dfp abs(Dfp x) {
        return x.abs();
    }
    // ---- wrappers that pass AllowedSolution ----
    private static void runCase(String name, FieldBracketingNthOrderBrentS
olver<Dfp> solver,
        RealFieldUnivariateFunction<Dfp> f, Dfp min, Dfp max, int maxEval) {
        runCaseAllowed(name, solver, f, min, max, maxEval, AllowedSolution.AN
Y_SIDE);
    }
    private static void runCaseAllowed(String name, FieldBracketingNthOrde
rBrentSolver<Dfp> solver,
        RealFieldUnivariateFunction<Dfp> f, Dfp min, Dfp max, int maxEval, A
llowedSolution side) {
        System.out.println("---- " + name + " ----");
        try {
            Dfp r = solver.solve(maxEval, f, min, max, side);

```

```

        System.out.printf("root=%s, |f(root)|=%s%n", r.toString(), abs(f.value(r)).toString());
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

private static void runCaseWithStart(String name, FieldBracketingNthOrderBrentSolver<Dfp> solver,
    RealFieldUnivariateFunction<Dfp> f, Dfp min, Dfp max, Dfp start, int
maxEval) {
    runCaseWithStartAllowed(name, solver, f, min, max, start, maxEval, AllowedSolution.ANY_SIDE);
}

private static void runCaseWithStartAllowed(String name, FieldBracketingNthOrderBrentSolver<Dfp> solver,
    RealFieldUnivariateFunction<Dfp> f, Dfp min, Dfp max, Dfp start, int
maxEval, AllowedSolution side) {
    System.out.println("---- " + name + " (with start) ----");
    try {
        Dfp r = solver.solve(maxEval, f, min, max, start, side);
        System.out.printf("root=%s, |f(root)|=%s%n", r.toString(), abs(f.value(r)).toString());
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getSimpleName() + " - "
+ t.getMessage());
    }
}

private static void expectThrow(String name, Runnable r, Class<?> expected) {
    System.out.println("---- " + name + " ----");
    try {
        r.run();
        check(name + " should throw " + expected.getSimpleName(), false);
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        check(name + " threw expected", t.getClass().equals(expected));
    }
}

public void Vtest() {
    System.out.println("=== FieldBrent (AllowedSolution fixed) – corrected VT1-VT4 ===");
}

```

```

FieldBracketingNthOrderBrentSolver<Dfp> SOLV_DEF = new FieldBracketin
gNthOrderBrentSolver<>(d(1e-12), d(1e-12),
    d(1e-15), 5);
FieldBracketingNthOrderBrentSolver<Dfp> SOLV_LOOSE = new FieldBracket
ingNthOrderBrentSolver<>(d(1e-12), d(1e-6),
    d(1e-30), 5);
// VT1: f == null → NullArgumentException (observed)
expectThrow("VT1 f==null -> NullArgumentException",
    () -> SOLV_DEF.solve(100, null, d(-
1.0), d(1.0), AllowedSolution.ANY_SIDE),
    org.apache.commons.math4.legacy.exception.NullArgumentException.class);
// VT2: bad interval – use equal bounds to deterministically hit
// NumberIsTooLargeException
// (some versions fold checks differently for reversed bounds; equal
bounds is
// always invalid)
RealFieldUnivariateFunction<Dfp> fLinear = x -> x.subtract(d(1.0));
expectThrow("VT2 min==max -> NumberIsTooLargeException",
    () -> SOLV_DEF.solve(100, fLinear, d(1.0), d(1.0), AllowedSolution.
ANY_SIDE),
    org.apache.commons.math4.legacy.exception.NumberIsTooLargeException
.class);
// VT3: invalid order < 2 – constructor throws NumberIsTooSmallExcept
ion
expectThrow("VT3 invalid order <2 -> NumberIsTooSmallException", () -
> {
    new FieldBracketingNthOrderBrentSolver<>(d(1e-12), d(1e-12), d(1e-
15), 1);
}, org.apache.commons.math4.legacy.exception.NumberIsTooSmallExceptio
n.class);
// VT4: invalid order > 5 – constructor throws NumberIsTooLargeExcept
ion
expectThrow("VT4 invalid order >5 -> NumberIsTooLargeException", () -
> {
    new FieldBracketingNthOrderBrentSolver<>(d(1e-12), d(1e-12), d(1e-
15), 50);
}, org.apache.commons.math4.legacy.exception.NumberIsTooLargeExceptio
n.class);
// VT5: not bracketed → NoBracketing
expectThrow("VT5 not bracketed -> NoBracketing", () -> {
    RealFieldUnivariateFunction<Dfp> f = x -> x.multiply(x).add(d(1.0));
// >0
    SOLV_DEF.solve(100, f, d(-3.0), d(2.0), AllowedSolution.ANY_SIDE);

```

```

    }, org.apache.commons.math4.legacy.exception.NoBracketingException.class);
    // VT6: endpoint root at min
    runCase("VT6 endpoint root @ min", SOLV_DEF, x -> x, d(0.0), d(1.0),
200);
    // VT7: endpoint root at max
    runCase("VT7 endpoint root @ max", SOLV_DEF, x -> x.subtract(d(1.0)),
d(0.0), d(1.0), 200);
    // VT8: start is root
    runCaseWithStart("VT8 start is root", SOLV_DEF, x -> x, d(-
1.0), d(0.0), d(0.0), 200);
    // VT9: start*min opposite sign → (min,start]
    runCaseWithStart("VT9 start*min opposite sign", SOLV_DEF, x -> x.subtract(d(0.5)), d(0.0), d(2.0), d(1.0), 300);
    // VT10: start*max opposite sign → [start,max)
    runCaseWithStart("VT10 start*max opposite sign", SOLV_DEF, x -> x.subtract(d(0.5)), d(-1.0), d(1.0), d(0.0),
300);
    // VT11: value-accuracy termination (sin root ~π in [3,4])
    runCase("VT11 value-
accuracy termination", SOLV_DEF, x -> x.sin(), d(3.0), d(4.0), 500);
    // VT12: step-size termination (looser abs acc)
    runCase("VT12 step-size termination", SOLV_LOOSE, x -> x.tanh(), d(-
1e-6), d(1e-6), 200);
    // VT13: bracket update left-leaning
    runCase("VT13 bracket update left", SOLV_DEF, x -> (x.subtract(d(0.2))
).multiply(x.subtract(d(1.5))), d(-0.5),
d(1.0), 300);
    // VT14: bracket update right-leaning
    runCase("VT14 bracket update right", SOLV_DEF, x -> (x.add(d(1.0))).m
ultiply(x.subtract(d(3.1))), d(2.6),
d(3.6), 300);
    // VT15: atan nonlinearity
    runCase("VT15 sharp nonlinearity", SOLV_DEF, x -> x.multiply(d(50.0))
.atan(), d(-0.5), d(0.5), 500);
    // VT16: clustered roots
    runCase("VT16 clustered roots", SOLV_DEF,
x -> (x.subtract(d(0.1))).multiply(x.subtract(d(0.11))).multiply(x.
subtract(d(0.5))), d(-0.2), d(0.3),
500);
    // VT17: near-linear
    runCase("VT17 near linear", SOLV_DEF, x -> (x.multiply(d(1e-
9))).add(x.subtract(d(0.7))), d(0.2), d(1.2), 400);
    // VT18: TooManyEvaluations (tiny budget)

```

```

    expectThrow("VT18 TooManyEvaluations (maxEval=1) -> MaxCountExceededE
xception", () -> {
        FieldBracketingNthOrderBrentSolver<Dfp> s = new FieldBracketingNthOr
derBrentSolver<>(d(1e-12), d(1e-12),
            d(1e-15), 5);
        RealFieldUnivariateFunction<Dfp> f = x -> x.cos().subtract(x); // ro
ot ~0.739 in [0,1]
        s.solve(1, f, d(0.0), d(1.0), AllowedSolution.ANY_SIDE);
    }, org.apache.commons.math4.legacy.exception.MaxCountExceededExceptio
n.class);
    // VT19: always positive -> NoBracketing
    expectThrow("VT19 no bracketing (always positive) -> NoBracketingExce
ption", () -> {
        RealFieldUnivariateFunction<Dfp> f = x -> x.multiply(x).add(d(0.5));
        // > 0 on [-1,2]
        SOLV_DEF.solve(200, f, d(-1.0), d(2.0), AllowedSolution.ANY_SIDE);
    }, org.apache.commons.math4.legacy.exception.NoBracketingException.cl
ass);
    // VT20: start inside, regular convergence
    runCaseWithStart("VT20 start inside, regular convergence", SOLV_DEF,
        x -> x.subtract(d(2.0)).multiply(x.add(d(1.0))).multiply(x.subtract
(d(5.0))), d(1.5), d(2.6), d(2.2),
        400);
    System.out.println("=== Done FieldBracketingNthOrderBrentSolver V-
method tests ===");
}
}
class FBNFT {
    private static final DfpField F = new DfpField(50);
    private static Dfp d(double v) {
        return F.newDfp(v);
    }
    private static Dfp abs(Dfp x) {
        return x.abs();
    }
    private static void log(String s) {
        System.out.println(s);
    }
    private static void runSolve(String name, FieldBracketingNthOrderBrent
Solver<Dfp> solver,
        RealFieldUnivariateFunction<Dfp> f, Dfp min, Dfp max, AllowedSolutio
n side, Integer maxEvalOpt) {
        System.out.println("---- " + name + " ----");
        try {

```



```

    int me = (maxEvalOpt == null ? 500 : maxEvalOpt);
    Dfp r = solver.solve(me, f, min, max, side);
    System.out.printf("root=%s, |f(root)|=%s%n", r, abs(f.value(r)));
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
}
}

private static void runSolveStart(String name, FieldBracketingNthOrder
BrentSolver<Dfp> solver,
    RealFieldUnivariateFunction<Dfp> f, Dfp min, Dfp max, Dfp start, AllowedSolution side, Integer maxEvalOpt) {
    System.out.println("---- " + name + " (with start) ----");
    try {
        int me = (maxEvalOpt == null ? 500 : maxEvalOpt);
        Dfp r = solver.solve(me, f, min, max, start, side);
        System.out.printf("root=%s, |f(root)|=%s%n", r, abs(f.value(r)));
    } catch (Throwable t) {
        System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
    }
}

// ===== random helpers (favor extreme-looking magnitudes) =====
private static double rU(Random r, double lo, double hi) {
    return lo + r.nextDouble() * (hi - lo);
}

private static double rMag(Random r) {
    // ~ 10^(U[-6, 308]) * U[0.5, 1.5], with random sign; clamp to safe range for Dfp
    // ops we use
    double e = rU(r, -6.0, 308.0);
    double m = Math.pow(10.0, Math.min(e, 308.0)) * (0.5 + r.nextDouble());
    return (r.nextBoolean() ? 1 : -1) * m;
}

private static AllowedSolution rSide(Random r) {
    switch (r.nextInt(5)) {
    case 0:
        return AllowedSolution.ANY_SIDE;
    case 1:
        return AllowedSolution.LEFT_SIDE;
    case 2:
        return AllowedSolution.RIGHT_SIDE;
    case 3:

```

```

        return AllowedSolution.BELOW_SIDE;
    default:
        return AllowedSolution.ABOVE_SIDE;
    }
}

public void Ftest() {
    long seed = 0xC0FFEE ^ System.nanoTime();
    Random rnd = new Random(seed);
    log("=== FieldBrent RANDOM FUZZ (20) - seed=0x" + Long.toHexString(seed).toUpperCase() + " ===");
    FieldBracketingNthOrderBrentSolver<Dfp> S_DEF = new FieldBracketingNthOrderBrentSolver<>(d(1e-12), d(1e-12),
        d(1e-15), 5);
    // 20 randomized cases
    for (int i = 1; i <= 20; i++) {
        int kind = rnd.nextInt(8);
        // choose an "extreme-ish" center and span
        double center = rMag(rnd); // can look like ±1.78E308, etc.
        double span = Math.abs(rMag(rnd)); // span magnitude
        // keep spans reasonable so functions like tanh/atan behave
        span = Math.max(1e-6, Math.min(span, 1e6));
        double a = center - span;
        double b = center + span;
        if (a > b) {
            double t = a;
            a = b;
            b = t;
        }
        // random start near the center
        double s = center + rU(rnd, -0.3 * span, 0.3 * span);
        AllowedSolution side = rSide(rnd);
        int maxEval = 100 + rnd.nextInt(900);
        // Build a function with a root near "center"
        RealFieldUnivariateFunction<Dfp> f;
        String fname;
        switch (kind) {
            case 0:
                // linear
                fname = "linear";
                final Dfp R0 = d(center);
                f = x -> x.subtract(R0);
                break;
            case 1:

```

```

    // quadratic shifted (odd about center: (x-center)*(x-
center) - eps)
    fname = "quadratic-ε";
    final Dfp R1 = d(center);
    final Dfp eps = d(rU(rnd, -1e2, 1e2));
    f = x -> x.subtract(R1).multiply(x.subtract(R1)).subtract(eps);
    break;
case 2:
    // tanh belly crossing at center
    fname = "tanh";
    final Dfp R2 = d(center);
    final Dfp k2 = d(rU(rnd, 1.0, 200.0));
    f = x -> x.subtract(R2).multiply(k2).tanh(); // tanh(k*(x-
R2)) crosses at R2
    break;
case 3:
    // sin around center with moderate freq
    fname = "sin";
    final Dfp w = d(rU(rnd, 1e-6, 3.0));
    final Dfp R3 = d(center);
    f = x -> (x.subtract(R3)).multiply(w).sin();
    break;
case 4:
    // atan scaled about center
    fname = "atan";
    final Dfp k4 = d(rU(rnd, 0.1, 100.0));
    final Dfp R4 = d(center);
    f = x -> (x.subtract(R4)).multiply(k4).atan();
    break;
case 5:
    // logistic shifted:  $\sigma(k*(x-center)) - 0.5$ 
    fname = "logistic";
    final Dfp k5 = d(rU(rnd, 0.1, 50.0));
    final Dfp R5 = d(center);
    f = x -> (x.subtract(R5)).multiply(k5).negate().exp().add(d(1)).rec
iprocal().subtract(d(0.5));
    break;
case 6:
    // rational-ish root:  $x - A/(1+B x + C x^2) \approx 0$  near center
    fname = "rational";
    final Dfp A6 = d(rMag(rnd));
    final Dfp B6 = d(rU(rnd, -1e-3, 1e-3));
    final Dfp C6 = d(rU(rnd, -1e-9, 1e-9));

```

```

    f = x -> x.subtract(A6.divide(d(1).add(B6.multiply(x)).add(C6.multiply(x).multiply(x))));
    break;
default:
    // cubic with tiny slope perturbation, root near center
    fname = "cubic-tilt";
    final Dfp R7 = d(center);
    final Dfp t7 = d(rU(rnd, -1e-6, 1e-6));
    f = x -> x.subtract(R7).pow(3).add(t7.multiply(x.subtract(R7)));
    break;
}
// Decide scenario: 70% bracketed, 15% no bracket, 10% equal bounds,
5% tiny
// budget
double p = rnd.nextDouble();
if (p < 0.70) {
    // bracket around [a,b] by ensuring the root is inside:
    // For monotone-ish forms, [a,b] already surrounds center.
    // For safety, shrink a bit so solver won't get crazy numbers.
    Dfp da = d(a), db = d(b), ds = d(s);
    runSolveStart(String.format("RFZ%02d %s bracketed", i, fname), S_DEF, f, da, db, ds, side, maxEval);
} else if (p < 0.85) {
    // no bracketing (shift interval away from center so signs match)
    double shift = span * (2.0 + rnd.nextDouble()); // push away from center
    double la = center + shift;
    double lb = la + span;
    Dfp da = d(la), db = d(lb);
    runSolve(String.format("RFZ%02d %s NOT bracketed", i, fname), S_DEF, f, da, db, side, maxEval);
} else if (p < 0.95) {
    // equal bounds (bad interval)
    double eq = center + rU(rnd, -0.4 * span, 0.4 * span);
    Dfp deq = d(eq);
    runSolve(String.format("RFZ%02d %s equal-bounds", i, fname), S_DEF, f, deq, deq, side, maxEval);
} else {
    // tiny budget – even if bracketed, try to trip MaxCountExceededException
    double sspan = Math.max(1e-6, span * 1e-3);
    Dfp da = d(center - sspan), db = d(center + sspan), ds = d(s);
    runSolveStart(String.format("RFZ%02d %s tiny-budget", i, fname), S_DEF, f, da, db, ds, side, 1);
}

```

```

    }
}
log("=== Done RANDOM FUZZ ===");
}
}
class FBNZ3 {
    private static final DfpField F = new DfpField(50);
    private static Dfp d(double v) {
        return F.newDfp(v);
    }
    private static Dfp abs(Dfp x) {
        return x.abs();
    }
    private static void check(String name, boolean cond) {
        System.out.printf("[%s] %s\n", cond ? "PASS" : "FAIL", name);
    }
    private static void expectThrow(String name, Runnable r, Class<?> expected) {
        System.out.println("---- " + name + " ----");
        try {
            r.run();
            check(name + " should throw " + expected.getSimpleName(), false);
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
            check(name + " threw expected", t.getClass().equals(expected));
        }
    }
    private static void runSolve(String name, FieldBracketingNthOrderBrent
Solver<Dfp> solver,
        RealFieldUnivariateFunction<Dfp> f, Dfp min, Dfp max, AllowedSolution
side, int maxEval) {
        System.out.println("---- " + name + " ----");
        try {
            Dfp r = solver.solve(maxEval, f, min, max, side);
            System.out.printf("root=%s, |f(root)|=%s\n", r, abs(f.value(r)));
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        }
    }
    private static void runSolveStart(String name, FieldBracketingNthOrder
BrentSolver<Dfp> solver,

```

```

    RealFieldUnivariateFunction<Dfp> f, Dfp min, Dfp max, Dfp start, AllowedSolution side, int maxEval) {
        System.out.println("---- " + name + " (with start) ----");
        try {
            Dfp r = solver.solve(maxEval, f, min, max, start, side);
            System.out.printf("root=%s, |f(root)|=%s%n", r, abs(f.value(r)));
        } catch (Throwable t) {
            System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
        }
    }

    public void Z3test() {
        System.out.println("=== FieldBrent - 20 scenarios from spec ===");
        // Default solver (maxOrder=5)
        FieldBracketingNthOrderBrentSolver<Dfp> SOLV_DEF = new FieldBracketingNthOrderBrentSolver<>(d(1e-12), d(1e-12), d(1e-15), 5);
        // 01) f=null -> NullArgument
        expectThrow("S01 f=null -> NullArgument", () -> SOLV_DEF.solve(100, null, d(-1), d(1), AllowedSolution.ANY_SIDE), org.apache.commons.math4.legacy.exception.NullArgumentException.class);
        // 02) min=null -> NullArgument
        RealFieldUnivariateFunction<Dfp> fLinear = x -> x; // simple
        expectThrow("S02 min=null -> NullArgument", () -> SOLV_DEF.solve(100, fLinear, null, d(1), AllowedSolution.ANY_SIDE), org.apache.commons.math4.legacy.exception.NullArgumentException.class);
        // 03) relAcc/absAcc/fAcc=null -> NullArgument (hit at construction)
        expectThrow("S03 relAcc null at ctor -> NullArgument", () -> {
            new FieldBracketingNthOrderBrentSolver<Dfp>(null, d(1e-12), d(1e-15), 5);
        }, org.apache.commons.math4.legacy.exception.NullArgumentException.class);
        // 04) maxOrder<2 -> InvalidOrder (NumberIsTooSmallException)
        expectThrow("S04 maxOrder<2 -> NumberIsTooSmallException", () -> {
            new FieldBracketingNthOrderBrentSolver<>(d(1e-12), d(1e-12), d(1e-15), 1);
        }, org.apache.commons.math4.legacy.exception.NumberIsTooSmallException.class);
        // 05) maxOrder>5 -> InvalidOrder (if your build enforces; else, just prove it

```

```

// works)
System.out.println("---- S05 maxOrder>5 ----");
try {
    FieldBracketingNthOrderBrentSolver<Dfp> s = new FieldBracketingNthOrderBrentSolver<>(d(1e-12), d(1e-12),
        d(1e-15), 6);
    Dfp root = s.solve(100, x -> x.sin(), d(3), d(4), AllowedSolution.ANY_SIDE);
    System.out.printf("constructed & solved with order=6, root=%s, |f(root)|=%s%n", root, root.sin().abs());
    check("S05 accepted >5 order (build allows it)", true);
} catch (org.apache.commons.math4.legacy.exception.NumberIsTooLargeException ok) {
    System.out.println("threw: " + ok.getClass().getName() + " - " + ok.getMessage());
    check("S05 threw (InvalidOrder>5)", true);
} catch (Throwable t) {
    System.out.println("threw: " + t.getClass().getName() + " - " + t.getMessage());
    check("S05 unexpected error", false);
}
// 06) min≥max → BadSequence (equal bounds, non-root) ->
// NumberIsTooLargeException
expectThrow("S06 min≥max (equal, non-root) -> NumberIsTooLargeException",
    () -> SOLV_DEF.solve(100, x -> x.subtract(d(1)), d(0.5), d(0.5), AllowedSolution.ANY_SIDE),
    org.apache.commons.math4.legacy.exception.NumberIsTooLargeException.class);
// 07) isNaN(min)∨isNaN(max) → NotANumber
Dfp NaN = F.getZero().divide(F.getZero());
expectThrow("S07 min is NaN -> NotANumber",
    () -> SOLV_DEF.solve(100, fLinear, NaN, d(1), AllowedSolution.ANY_SIDE),
    org.apache.commons.math4.legacy.exception.NotANumberException.class);
// 08) ¬bracketed → NoBracketing (use always-positive f on [-5,-3])
expectThrow("S08 not bracketed -> NoBracketing", () -> {
    RealFieldUnivariateFunction<Dfp> f = x -> x.multiply(x).add(d(1)); /
/ >0
    SOLV_DEF.solve(100, f, d(-5), d(-3), AllowedSolution.ANY_SIDE);
}, org.apache.commons.math4.legacy.exception.NoBracketingException.class);
// 09) yMin=0 → r=min (endpoint root)

```

```

runSolve("S09 yMin=0 -> r=min", SOLV_DEF, x -> x, d(0), d(1), Allowed
Solution.ANY_SIDE, 200);
// 10) yMax=0 -> r=max (endpoint root)
runSolve("S10 yMax=0 -> r=max", SOLV_DEF, x -> x.subtract(d(0)), d(-
1), d(0), AllowedSolution.ANY_SIDE, 200);
// 11) inInterval(start) & yStart=0 -> r=start
runSolveStart("S11 start in interval & yStart=0 -> r=start", SOLV_DEF
, x -> x, d(-1), d(1), d(0),
    AllowedSolution.ANY_SIDE, 200);
// 12) start vs min: opposite sign -> r∈(min,start]
// min=-6, start=-2, choose max=2, f(x)=x+5 has root at -
5 ∈ (min,start]
runSolveStart("S12 start vs min opposite sign", SOLV_DEF, x -> x.add(
d(5)), d(-6), d(2), d(-2),
    AllowedSolution.ANY_SIDE, 300);
// 13) start vs max: opposite sign -> r∈[start,max)
// start=-4, max=-3, choose min=-10, f(x)=x+4 has root at -
4 ∈ [start,max)
runSolveStart("S13 start vs max opposite sign", SOLV_DEF, x -> x.add(
d(4)), d(-10), d(-3), d(-4),
    AllowedSolution.ANY_SIDE, 300);
// 14) interp branch: |f(xNew)|≤fAcc -> r=xNew
// Make fAcc large so value-accuracy triggers quickly in [-8,4]
FieldBracketingNthOrderBrentSolver<Dfp> SOLV_FACC = new FieldBracketi
ngNthOrderBrentSolver<>(d(1e-12), d(1e-12),
    d(1.0), 4);
runSolve("S14 interpolation/value-
accuracy termination", SOLV_FACC, x -> (x.add(d(1))).multiply(x.add(d(3
))),
    d(-8), d(4), AllowedSolution.ANY_SIDE, 300);
// 15) interp: (b-a)≤tol(xNew) -> r=xNew
// Use relAcc=0, absAcc=7 on [0,7]; any interior step should satisfy
step
// tolerance fast
FieldBracketingNthOrderBrentSolver<Dfp> SOLV_STEP = new FieldBracketi
ngNthOrderBrentSolver<>(d(0.0), d(7.0),
    d(0.0), 5);
runSolve("S15 step-
size termination", SOLV_STEP, x -> x, d(0), d(7), AllowedSolution.ANY_S
IDE, 200);
// 16) bracketed & candidateNotUsable -> xSafe (we just ensure converg
ence on a
// benign bracket)
runSolve("S16 bracketed, safe step fallback possible", SOLV_DEF,

```



```

    x -> (x.subtract(d(1))).multiply(x.subtract(d(2))), d(0), d(2.5), AllowedSolution.ANY_SIDE, 400);
    // 17) iterative narrowing  $\rightarrow r \in [a', b']$  (typical convergence with mid update)
    runSolve("S17 iterative narrowing", SOLV_DEF, x -> x, d(-1), d(2), AllowedSolution.ANY_SIDE, 300);
    // 18) evaluations > maxEval  $\rightarrow$  TooManyEvaluations (MaxCountExceededException)
    expectThrow("S18 TooManyEvaluations (maxEval=0) -> MaxCountExceededException", () -> {
        RealFieldUnivariateFunction<Dfp> f = x -> x.cos().subtract(x);
        SOLV_DEF.solve(0, f, d(0), d(1), AllowedSolution.ANY_SIDE);
    }, org.apache.commons.math4.legacy.exception.MaxCountExceededException.class);
    // 19) interp: order=maxOrder(5),  $|f(x_{\text{New}})| \leq f_{\text{Acc}} \rightarrow r = x_{\text{New}}$ 
    FieldBracketingNthOrderBrentSolver<Dfp> SOLV_ORDER5_FACC = new FieldBracketingNthOrderBrentSolver<>(d(1e-12),
        d(1e-12), d(1.0), 5);
    runSolve("S19 order=5, value-accuracy", SOLV_ORDER5_FACC, x -> (x.subtract(d(1))).multiply(x.add(d(1)))), d(-2),
        d(2), AllowedSolution.ANY_SIDE, 400);
    // 20) candidateNotUsable &  $(b-a) \leq \text{tol}(x_{\text{Safe}}) \rightarrow r = x_{\text{Safe}}$ 
    // Use big absAcc=2 on  $[-2, 0]$ , root at -1
    FieldBracketingNthOrderBrentSolver<Dfp> SOLV_SAFE_STEP = new FieldBracketingNthOrderBrentSolver<>(d(0.0),
        d(2.0), d(0.0), 5);
    runSolve("S20 fallback + step-size", SOLV_SAFE_STEP, x -> x.add(d(1)), d(-2), d(0), AllowedSolution.ANY_SIDE,
        200);
    System.out.println("=== Done 20 scenarios ===");
}
}
class FBNtest {
    public static void main(String[] args) {
        // VT
        FBNVT t1 = new FBNVT();
        t1.Vtest();
        // FT
        FBNFT t2 = new FBNFT();
        // t2.Ftest();
        // Z3
        FBNZ3 t3 = new FBNZ3();

```

```
// t3.Z3test();
}
}
```

● Coverage report

1. Test case number: 20
2. V-Method testing branch coverage: 83.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
FieldBracketingNthOrderBrentSolver.j	57.6 %	57	42	99
> FBNFT	0.0 %	0	29	29
> FBNZ3	0.0 %	0	2	2
FieldBracketingNthOrderBrentSolv	83.3 %	55	11	66
● solve(int, RealFieldUnivariateFur	81.0 %	47	11	58
● FieldBracketingNthOrderBrentSi	100.0 %	2	0	2
■ guessX(T, T[], T[], int, int)	100.0 %	6	0	6
● solve(int, RealFieldUnivariateFur		0	0	0
> FBNVT	100.0 %	2	0	2
> FBNtest		0	0	0

3. V-Method testing instruction coverage: 93.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
FieldBracketingNthOrderBrentSolver.j	45.8 %	1,237	1,462	2,699
> FBNZ3	1.3 %	9	675	684
> FBNFT	1.3 %	9	662	671
> FBNtest	83.3 %	15	3	18
> FBNVT	88.0 %	550	75	625
FieldBracketingNthOrderBrentSolv	93.3 %	654	47	701
● solve(int, RealFieldUnivariateFur	95.1 %	525	27	552
● FieldBracketingNthOrderBrentSi	100.0 %	34	0	34
■ guessX(T, T[], T[], int, int)	100.0 %	80	0	80
● solve(int, RealFieldUnivariateFur	100.0 %	15	0	15

4. Fuzz testing branch coverage: 66.7%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
FieldBracketingNthOrderBrentSolver.j	68.7 %	68	31	99
> FBNVT	0.0 %	0	2	2
> FBNZ3	0.0 %	0	2	2
FieldBracketingNthOrderBrentSolv	66.7 %	44	22	66
● FieldBracketingNthOrderBrentSi	50.0 %	1	1	2
● solve(int, RealFieldUnivariateFur	63.8 %	37	21	58
■ guessX(T, T[], T[], int, int)	100.0 %	6	0	6
● solve(int, RealFieldUnivariateFur		0	0	0
> FBNFT	82.8 %	24	5	29
> FBNtest		0	0	0

5. Fuzz testing instruction coverage: 83.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
FieldBracketingNthOrderBrentSolver.j	43.6 %	1,177	1,522	2,699
> FBNZ3	1.3 %	9	675	684
> FBNVT	1.4 %	9	616	625
FieldBracketingNthOrderBrentSolv	83.3 %	584	117	701
● FieldBracketingNthOrderBrentSi	73.5 %	25	9	34
● solve(int, RealFieldUnivariateFur	84.1 %	464	88	552
■ guessX(T, T[], T[], int, int)	100.0 %	80	0	80
● solve(int, RealFieldUnivariateFur	100.0 %	15	0	15
> FBNtest	83.3 %	15	3	18
> FBNFT	83.5 %	560	111	671

6. Z3 branch coverage: 81.8%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
FieldBracketingNthOrderBrentSolver.j	56.6 %	56	43	99
> FBNFT	0.0 %	0	29	29
> FBNVT	0.0 %	0	2	2
FieldBracketingNthOrderBrentSolv	81.8 %	54	12	66
● solve(int, RealFieldUnivariateFur	79.3 %	46	12	58
● FieldBracketingNthOrderBrentSi	100.0 %	2	0	2
■ guessX(T, T[], T[], int, int)	100.0 %	6	0	6
● solve(int, RealFieldUnivariateFur		0	0	0
> FBNZ3	100.0 %	2	0	2
> FBNtest		0	0	0

7. Z3 instruction coverage: 92.4%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
FieldBracketingNthOrderBrentSolver.j	44.1 %	1,190	1,509	2,699
> FBNFT	1.3 %	9	662	671
> FBNVT	1.4 %	9	616	625
> FBNZ3	74.4 %	509	175	684
> FBNtest	83.3 %	15	3	18
FieldBracketingNthOrderBrentSolv	92.4 %	648	53	701
● solve(int, RealFieldUnivariateFur	94.0 %	519	33	552
● FieldBracketingNthOrderBrentSi	100.0 %	34	0	34
■ guessX(T, T[], T[], int, int)	100.0 %	80	0	80
● solve(int, RealFieldUnivariateFur	100.0 %	15	0	15